

거의 다 났습니다. 지금 가입해서 Notion으로 구축을 시작하세요.

가입 또는 로그인하기

3주차

≡ 설명

기본 과제 설명 글

📎 파일과 미디어

기초반 3주차.pdf

기본 과제 설명

회합에서는 파라미터를 업데이트할 때 gradient에 learning rate를 곱하는 간단한 방식만 살펴봤다. 실제로는 SGD, Momentum, RMSProp, Adam 등 다양한 업데이트 기법이 존재한다. 기본 과제를 통해 위의 4가지 기법에 대해서 배워보도록 하자.

이해하기 어려운 부분이 있다면 학술부장 김성은이나 본인의 멘토 분에게 질문을 하셔도 되고, 제미나이나 챗지피티 등의 인공지능에게 질문하셔도 좋습니다.

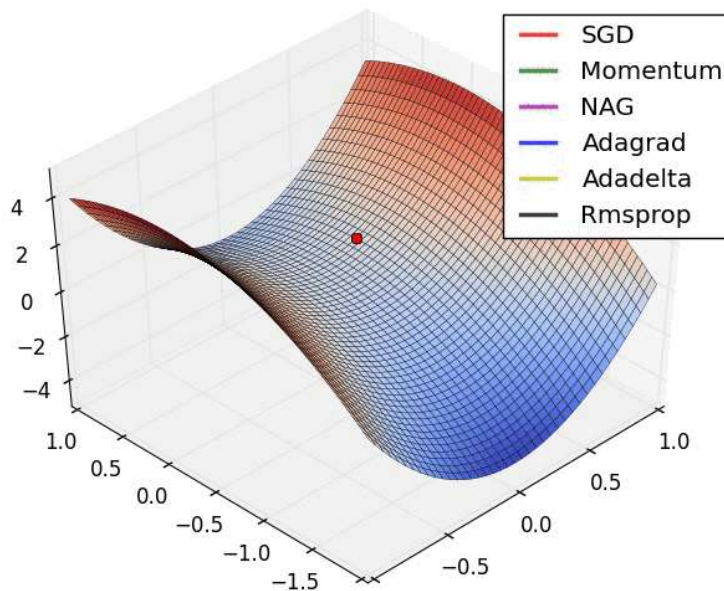
Optimizer(옵티마이저)란?

다층 퍼셉트론(이하 neural network)은 역전파를 통해 각 파라미터의 편미분 값을 구하고, 이를 통해 파라미터를 어떤 방향으로 얼마나 업데이트 할지 구한다. optimizer는 neural network의 파라미터를 업데이트하는 방식을 결정한다.

SGD, Momentum, RMSProp, Adam 등 여러 종류의 optimizer가 있고, 각 optimizer는 업데이트 방식의 직관적 의미와 파라미터 수정 방법이 다르다. 아래 설명을 꼼꼼히 읽고 다양한 optimizer를 수식과 코드를 통해 이해 해보도록 하자.

모든 모델에 적용되는 최고의 optimizer는 존재하지 않는다. 따라서 모델에 다양한 optimizer를 적용해보고 최고의 성능을 보이는 것으로 결정하는 것이 최선이다. (하지만 실제로 Adam의 경우 대부분의 학습에서 좋은 성능을 보인다. 기본 과제의 목표는 Adam을 이해 하는 것이다!!)

Optimizer 종류



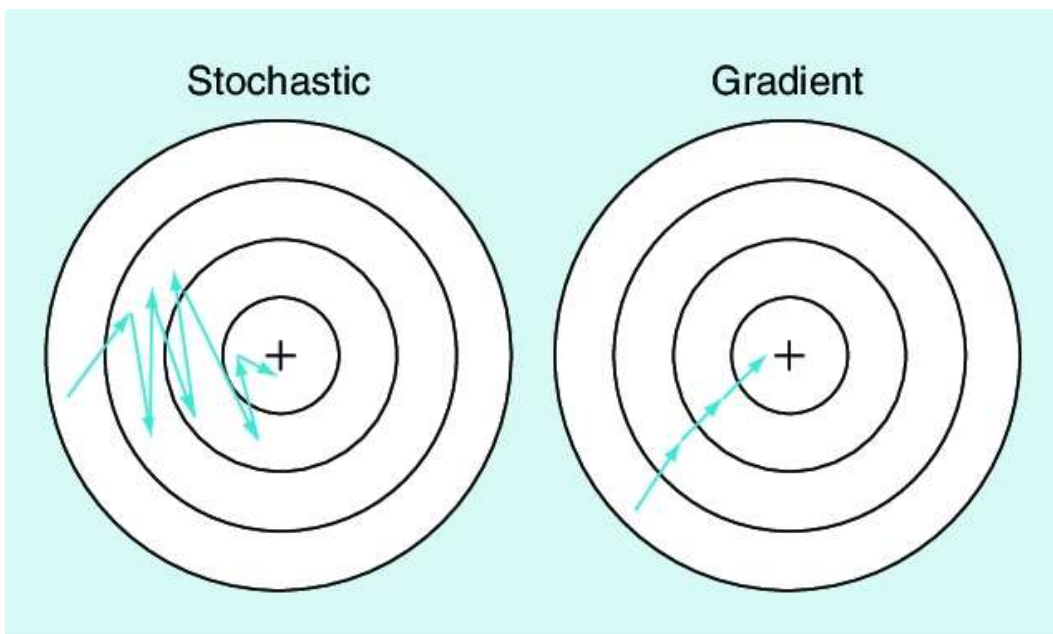
다양한 경사하강법

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

위는 경사하강법을 이용해서 가중치를 조절하는 가장 기본적인 방법이다.

이런 방법을 Batch Gradient Descent라고 부른다. N개의 전체 데이터에 대한 계산 결과를 모아서 한 번에 파라미터 업데이트를 하기 때문에 데이터의 양이 많아지면 학습 시간이 증가한다는 단점이 있다.

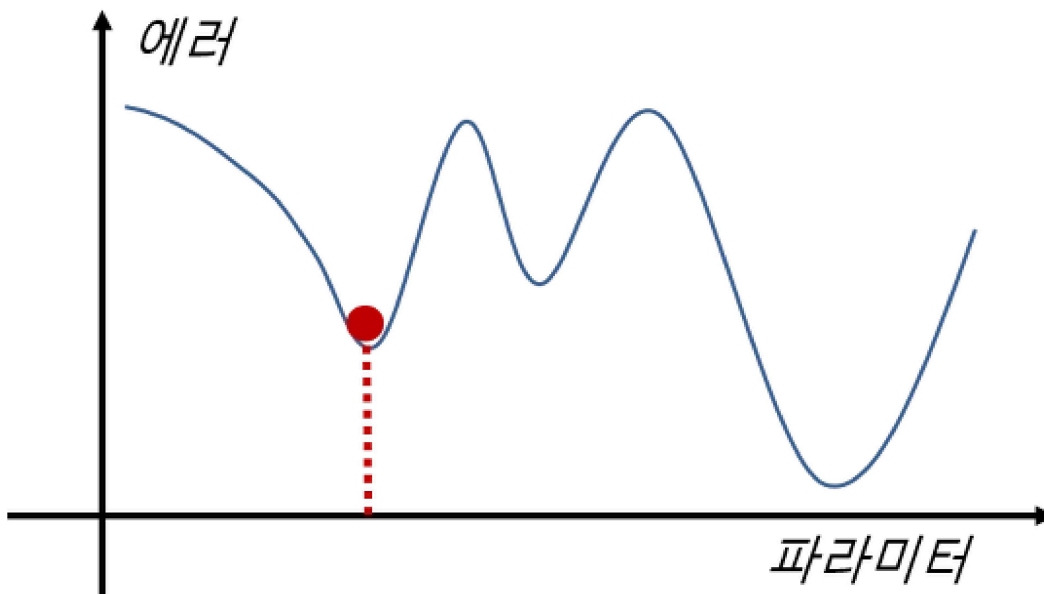
이를 해결하기 위해서 등장한 방법이 SGD(Stochastic gradient descent, 확률적 경사하강법)이다. 확률적으로 1개의 데이터를 뽑아서 loss 값을 계산하고 가중치를 업데이트 하는 방식이다. 적은 메모리와 적은 시간으로 모델을 학습할 수 있다는 장점이 있지만, 아래 그림과 같이 학습이 불안정하고 노이즈가 심하다는 단점이 있다.



하나의 데이터가 모든 데이터의 오차를 반영할 수 없기 때문에 등장한 것이 Mini-Batch Gradient Descent다. 데이터 1개가 아니라 적당한 크기(batch size)의 데이터를 활용해서 모델을 업데이트 한다. BGD 보다 빠르고 SGD 보다 안정적이라는 장점이 있다.

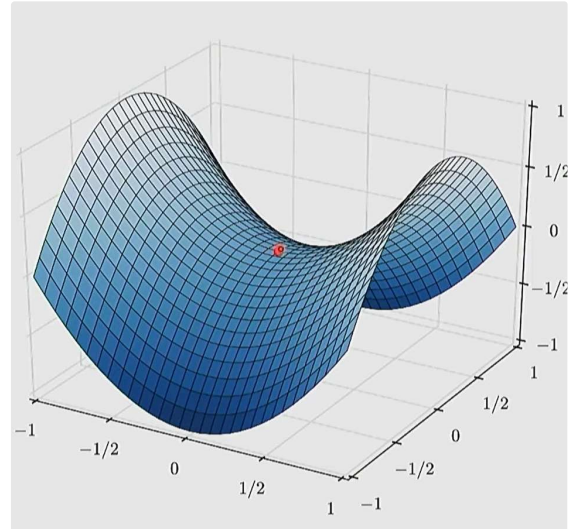
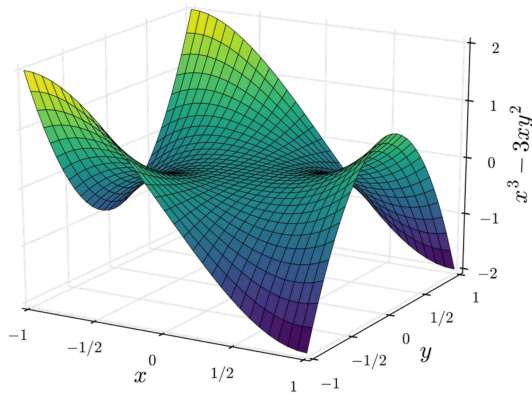
	Batch Gradient Descent	Stochastic Gradient Descent	Mini-Batch Gradient Descent
Batch Size	N개의 전체 데이터	N개의 전체 데이터 중 1개	N개의 전체 데이터 중 M개($1 < M < N$)

이런 경사하강법에 공통적인 단점이 존재한다.

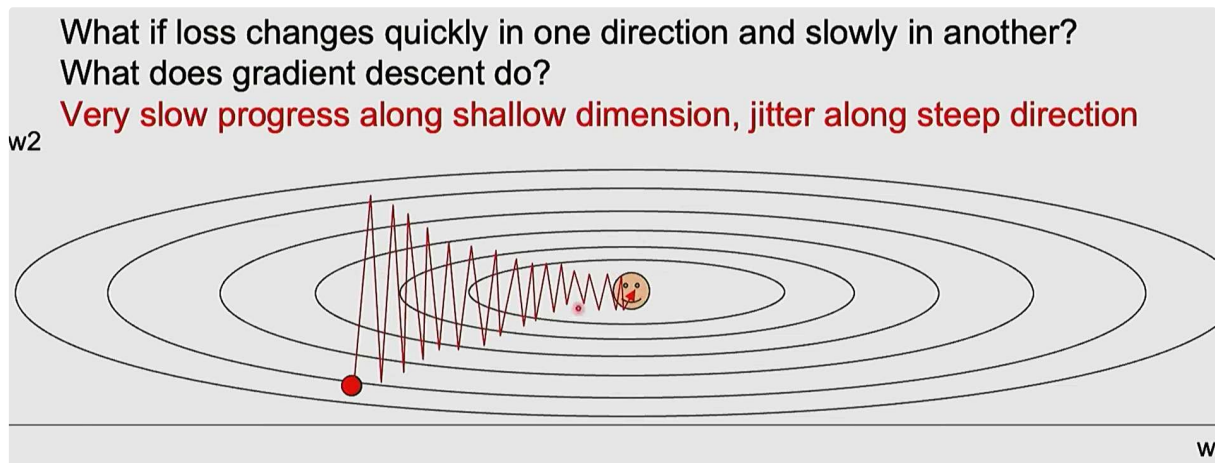


위의 사진에서 우리가 찾고자 하는 최솟값은 오른쪽 아래의 볼록한 부분이다. 그러나 경사하강법으로 업데이트하면 왼쪽의 볼록한 부분에 멈출 수 있다. 이런 문제를 Local Minima 문제라고 한다. Global Minima를 찾아야 하지만 Local Minima에 갇히게 되는 것이다.

또한 2차원보다 차원이 많은 고차원일 때는 Saddle Point(안장점) 문제가 발생한다.



위의 두 사진은 안장점을 나타낸다. 안장점이 존재한다는 것은 어느 방향에서는 극댓값이지만, 어느 방향에서는 극솟값인 위치가 있는 것이다. 어느 방향으로든 편미분 값이 0 이기 때문에 경사하강법으로 업데이트 될 수 없다.



또 다른 문제점은 기울기가 작은 방향(완만한 방향)으로는 수렴하는 속도가 느린 반면, 기울기가 큰 방향(가파른 방향)으로는 스텝 사이즈가 너무 커지기 때문에 수렴이 지그재그로 되면서 불안정하다는 점이다.

이와 같은 경사하강법의 한계를 극복하기 위해 등장한 방법들을 계속 살펴볼 것이다.

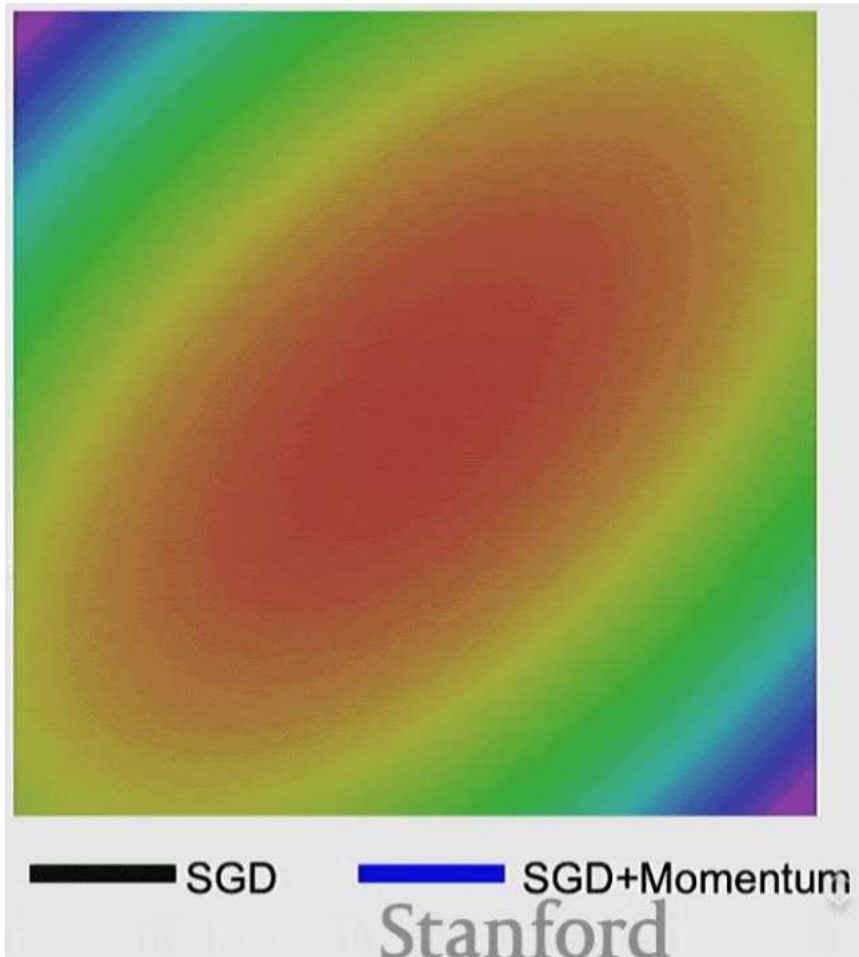
Momentum(모멘텀)

모멘텀은 쉽게 말해서 '관성'이다. 관성을 도입하기 위해 $\frac{\partial L}{\partial w}$ 로 바로 파라미터를 업데이트 하는 것이 아니라 v (속도)를 통해 간접적으로 업데이트하게 된다. 과거의 속도를 유지하기 때문에 local minima를 빠져나올 가능성이 높아진다.

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \eta \frac{\partial L}{\partial \mathbf{W}}$$

$$\mathbf{W} \leftarrow \mathbf{W} + \mathbf{v}$$

α, η 는 모델 학습 전에 설정해줘야 하는 하이퍼파라미터다. η 는 learning rate고 보통 0.01이나 0.001 같이 작은 값을 취한다. α 는 Momentum Coefficient(모멘텀 계수)로 이전 스텝에서 이동하던 방향(속도)을 얼마나 유지할지 결정하는 관성의 크기다. 보통 0.9를 많이 사용하고 0.99를 사용하기도 한다. 즉, 이전 스텝에서 움직이던 관성을 90% 정도 살리고, 거기에 현재 발바닥으로 느껴지는 기울기를 반영해서 다음 발걸음을 내딛겠다는 직관적인 의미를 담고 있다.



위의 이미지에서 알 수 있듯이 SGD는 noisy하고 수렴이 불안정한 반면, SGD에 Momentum을 도입하면 기울기 방향으로 빠르고 안정적으로 수렴하는 것을 볼 수 있다.

RMSProp

RMSProp은 decay 개념을 통해서 각 원소의 과거 기울기 크기(가파름 정도)를 반영해서 원소 단위로 기울기를 조절한다. 원소 단위로 조절한다는 것은 벡터의 각 원소 별로 업데이트 되는 정도가 다르다는 의미다.

RMSProp을 수행하는 간단한 코드를 살펴보자.

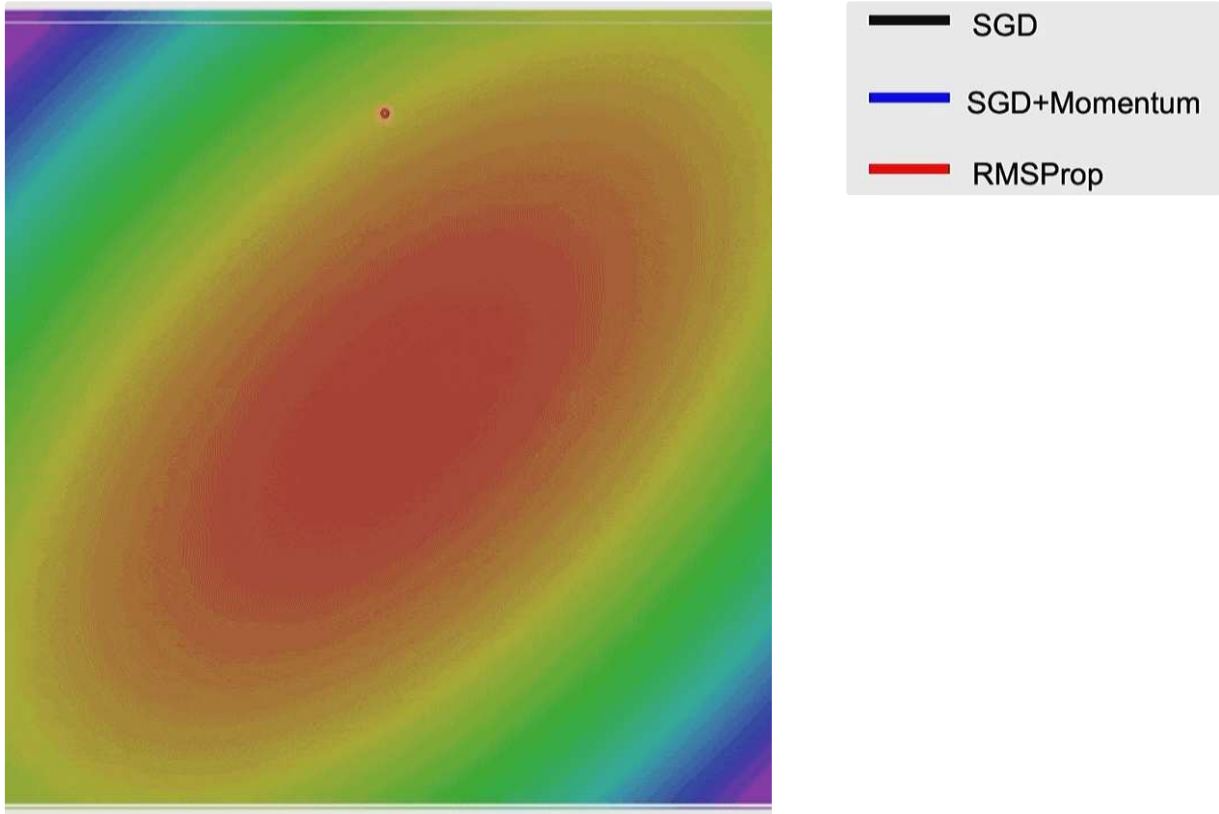
```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

`grad_squared` 는 기울기의 제곱을 기록하며 이전 기울기의 제곱에 현재 기울기의 제곱을 반영한 누적값이다. 제곱하는 이유는 방향은 무시하고 오직 기울기가 얼마나 가팔랐는지, 그 크기만 양수로 누적하기 위해서다. `np.sqrt(grad_squared)` 로 나누는데, 경사가 가팔라서 `grad_squared` 값이 컸다면 분모가 커지므로 업데이트 크기가 줄어들어서 조심조심 내려가게 된다. 반대로 경사가 완만해서 `grad_squared` 값이 작았다면 분모가 작아지므로 업데이트 크기가 커지게 돼서 성큼성큼 내려가게 된다.

`decay_rate` (감쇄율)은 이전 기울기의 제곱과 현재 기울기의 제곱을 어떤 비율로 반영할지 결정하는 상수로, 보통 0.9나 0.99 같은 값을 사용한다. 값이 클수록 과거 기울기 기록을 중요시 여긴다는 의미다.

참고로 `1e-7` 은 10^{-7} 이라는 의미로 분모가 0이 되는 것을 막기 위해서 더해주는 값이다.

그런데 왜 decay 라는 개념을 도입했을까? RMSProp 이전에 나온 Adagrad라는 기법은 과거의 모든 기울기 제곱을 누적(decay 없이 단순 덧셈)했고, 학습이 길어지면 분모가 굉장히 커져서 학습률이 0에 수렴해버려서 더 이상 업데이트가 안 되는 치명적인 단점이 있었기 때문이다.



위의 이미지를 통해 3가지 optimizer의 특징을 파악할 수 있다.

```
grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
```

위의 코드와 대응되는 실제 수식은 다음과 같다.

$$G(t) = \gamma G(t-1) + (1 - \gamma) \left(\frac{\partial}{\partial w(i)} \text{Cost}(w(i)) \right)^2$$

$G(t)$ 와 $G(t-1)$ 은 각각 현재 스텝(t)과 이전 스텝($t-1$)까지 누적된 기울기 제곱의 합이며, 코드에서의 `grad_squared`에 대응된다. γ 는 `decay_rate`에 해당하며 과거 기록이 서서히 잊히도록 해준다. $\frac{\partial}{\partial w(i)} \text{Cost}(w(i))$ 은 수식이 조금 무섭게 생겼지만, 그냥 현재 위치에서의 기울기라는 의미로 코드에서 `dx`에 해당한다. $(\dots)^2$ 으로 기울기를 제공하는 이유는 앞서 말했듯이 방향(음수/양수)은 무시하고 오직 경사가 얼마나 가팔랐는지만 크기로 기록하기 위해서이고, 코드에서 `dx * dx`에 해당한다.

```
x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

그리고 위의 코드와 대응되는 실제 수식은 다음과 같다.

$$V(t+1) = W(t) - \alpha * \frac{1}{\sqrt{G(t) + \epsilon}} * \frac{\partial}{\partial w(i)} Cost(w(i))$$

$W(t+1)$ 과 $W(t)$ 는 각각 다음 파라미터 값과 현재 파라미터 값을 의미한다. α 는 `learning_rate`를 의미한다. ϵ 는 분모가 0이 되어 에러가 나는 것을 막기 위해 더해주는 작은 숫자고, 코드의 `1e-7`에 해당한다. $\frac{1}{\sqrt{G(t) + \epsilon}}$ 은 보폭을 조절해주는 부분으로 과거의 경사가 가팔라서 분모의 $G(t)$ 가 크다면 전체 분수 값은 작아져서 보폭을 줄여주고, 과거의 경사가 완만해서 분모의 $G(t)$ 가 작다면 전체 분수 값은 커져서 보폭을 늘려준다. 코드에서 `1 / np.sqrt(grad_squared + 1e-7)`에 해당한다.

EMA(Exponential Moving Average, 지수 이동 평균)

RMSProp의 `grad_squared` 업데이트 방식은 EMA를 따른다. 즉, EMA는 optimizer는 아니고 RMSProp과 Adam을 이해하는 데 도움이 되는 수학 개념이다. 궁금한 사람은 읽어보도록 하자.

```
first_moment = beta1 * first_moment + (1 - beta1) * dx
```

위의 코드가 EMA를 표현한 코드이고 수식으로 표현하면 다음과 같다.

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t$$

- V_t : 현재 스텝(t)의 지수 이동 평균값(코드에서 업데이트된 `first_moment`에 해당)
- V_{t-1} : 이전 스텝(t-1)까지의 지수 이동 평균값(코드에서 업데이트 되기 전 `first_moment`에 해당)
- θ_t : 현재 스텝에서 새로 들어온 데이터(코드에서 현재 기울기 `dx`에 해당)
- β : 0과 1 사이의 하이퍼파라미터(코드에서 `beta1`에 해당, 보통 0.9)

직관적으로 이해해보면 (이전까지의 기억 × 유지할 비율) + (새로운 데이터 × 반영할 비율)의 형태다.

그럼 왜 지수라는 이름이 붙은 것일까? V_{t-1} 자리에 이전 스텝의 공식을 대입해서 식을 전개해보면 알 수 있다.

$$\begin{aligned}
 V_t &= \beta V_{t-1} + (1 - \beta)\theta_t \\
 &= \beta(\beta V_{t-2} + (1 - \beta)\theta_{t-1}) + (1 - \beta)\theta_t \\
 &= (1 - \beta)\theta_t + (1 - \beta)\beta\theta_{t-1} + \beta^2 V_{t-2}
 \end{aligned}$$

이 과정을 계속 반복하면, 현재 V_t 에 각 과거 데이터(θ)가 미치는 영향력은 다음과 같아진다.

- 현재 데이터 θ_t 의 가중치: $(1 - \beta)$
- 1-스텝 전 데이터 θ_{t-1} 의 가중치: $(1 - \beta)\beta$
- 2-스텝 전 데이터 θ_{t-2} 의 가중치: $(1 - \beta)\beta^2$
- k-스텝 전 데이터 θ_{t-k} 의 가중치: $(1 - \beta)\beta^k$

β 가 0.9 일 때, 예전 데이터 일수록 가중치에 $0.9^1, 0.9^2, 0.9^3, \dots$ 처럼 β 의 거듭제곱이 곱해지게 된다. 값이 1보다 작기 때문에 제공할수록 0으로 수렴하게 되고, 과거 데이터 일수록 가중치가 지수적으로 감소하기 때문에 이런 이름이 붙은 것이다.

딥러닝 optimizer에서 EMA를 사용하는 이유는 다음과 같다.

- 과거 데이터를 단순히 같은 비율로 더하게 되면 학습 후반부에 아주 옛날의 무의미한 기울기가 현재의 업데이트를 방해하게 된다. 따라서 EMA를 사용하면 현재의 기울기에 민감하게 반응하면서도 과거의 관성을 부드럽게 이어갈 수 있다.
- 과거의 모든 θ 값을 배열로 저장해 둘 필요 없이, 직전 스텝의 결과 값인 V_{t-1} 변수만 메모리에 넣고 있으면 되기 때문에 파라미터가 수억 개인 딥러닝 모델에서 큰 장점을 가진다.

Adam

드디어 우리의 목표인 Adam이다. 딥러닝에서 가장 유용하고 널리 쓰이는 optimizer이며, 모르겠으면 일단 Adam을 쓰면 된다고 말할 정도로 강력한 방식이다.

Adam은 간단히 말하면 Momentum + RMSProp이다.

아래의 간단한 코드를 통해 Adam을 이해해보자.(후에 설명하겠지만 허점이 있는 코드다.)

```
# 허점이 있는 코드
first_moment = 0
second_moment = 0
while True:
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    # Momentum
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    # RMSProp
    x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7)
    # RMSProp
```

`first_moment` 는 Momentum에 대응되고, `second_moment` 는 RMSProp에서의 decay 개념에 대응된다. 따라서 `x` 를 업데이트 하는 코드의 분자는 momentum을, 분모는 RMSProp을 활용하고 있다.

Momentum과 RMSProp의 장점을 합쳐서 만든 Adam은 두 moment가 과거 기록을 유지하되, 서서히 잊히도록 만들어서 학습이 관성을 가지도록 만든다. 그리고 가파른 곳에서는 조심스럽게, 완만한 곳에서는 과감하게 파라미터를 업데이트 한다.

`beta1` , `beta2` 는 1에 가깝게 설정(0.9, 0.999 등)되는 상수다.

그런데 이 코드에는 한 가지 근본적인 원인에서 비롯되는 두 가지 문제점이 있다.

`first_moment` 와 `second_moment` 를 0으로 초기화 하는데, `beta1` , `beta2` 는 1에 가까운 값을 가지기 때문에, 학습 초기에 두 moment 값이 0에 가깝게 편향(bias) 되는 것이 원인이다.

첫 번째 문제는 0으로의 편향 때문에 올바른 이동 방향이나 크기를 계산하지 못하고 의도치 않은 방향으로 업데이트를 하게 된다는 것이다.

두 번째 문제는 분자와 분모의 편향 정도가 달라서 스텝 사이즈(step size)가 비정상적으로 커진다는 점이다. `first_moment` 을 `np.sqrt(second_moment)` 로 나눌 때, 분모의 `beta2` (보통 0.999로 설정)가 분자의 `beta1` (보통 0.9로 설정)보다 1에 더 가깝기 때문에 분모가 0에 더 가깝게 줄어든다. 따라서 초기에 의도했던 스텝 사이즈보다 더 큰 스텝을 밟게 되어서 학습이 크게 튕어버린다.

- 분자의 편향 정도: `beta1` 은 보통 0.9를 쓴다. 첫 번째 스텝에서 분자(`first_moment`)는 실제 기울기(`dx`)의 $(1 - 0.9) = 0.1$ 배로 줄어든다.
- 분모의 편향 정도: `beta2` 는 보통 0.999를 쓴다. 분모에 루트가 씌워져 있으므로, 첫 번째 스텝에서 분모(`np.sqrt(second_moment)`)는 $\sqrt{1 - 0.999} = \sqrt{0.001} \approx 0.0316$ 배로 줄어든다.

따라서 업데이트 식은 $\frac{0.1}{0.0316} \approx 3.16$ 이 된다. 초기에 의도했던 `learning_rate` 보다 약 3.16배 더 큰 스텝 사이즈로 과도하게 업데이트 되어 버리는 문제가 발생하는 것이다. 그러므로 코드에 `first_unbias` , `second_unbias` 를 추가하여 아래와 같이 바꿔야 한다.

```
# 정확한 코드
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx # Momentum
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx # RMSProp
    first_unbias = first_moment / (1 - beta1 ** t) # Bias correction
    second_unbias = second_moment / (1 - beta2 ** t) # Bias correction
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7) # RMSProp
```

t 는 타임 스텝(time step)으로 학습이 얼마나 진행됐는지를 의미한다. 학습 초기에는 $(1 - \text{beta1} ** t)$ 와 $(1 - \text{beta2} ** t)$ 의 값이 0에 가깝다. `first_moment`와 `second_moment`에 각각 대응하는 `first_unbias`와 `second_unbias`를 통해 두 moment 값을 원래 기울기가 가져야 할 크기로 보정함으로써 0으로 편향되는 문제를 해결한다. 또한 학습이 진행될수록 $(1 - \text{beta1} ** t)$ 와 $(1 - \text{beta2} ** t)$ 의 값이 1에 가까워지기 때문에 unbias가 식에 미치는 영향은 점점 미미해진다.

이제 Adam에 대한 이해는 끝났다!

참고로 많은 모델에 적용되는 좋은 Adam 하이퍼파라미터 설정이 있는데 `beta1` = 0.9, `beta2` = 0.999, `learning_rate` = 1e-3 또는 5e-4로 하는 것이다.

마지막으로 Adam 수식을 제시하면서 글을 마친다. RMSProp과 유사하니 쉽게 이해할 수 있을 것이다.

$$M(t) = \beta_1 M(t-1) + (1 - \beta_1) \frac{\partial}{\partial w(t)} \text{Cost}(w(t))$$

$$V(t) = \beta_2 V(t-1) + (1 - \beta_2) \left(\frac{\partial}{\partial w(i)} \text{Cost}(w(i)) \right)^2$$

$$\widehat{M}(t) = \frac{M(t)}{1 - \beta_1^t} \quad \widehat{V}(t) = \frac{V(t)}{1 - \beta_2^t}$$

$$W(t+1) = W(t) - \alpha * \frac{\widehat{M}(t)}{\sqrt{\widehat{V}(t)} + \epsilon}$$

기본 과제

위의 글을 모두 읽고 이해한 뒤에, 제시된 11문제를 풀어주세요. 학술후장 김성은에게 개인 카톡으로 11문제에 대한 답을 보내주시면 과제 제출로 인정됩니다. 오답이 많다고 해서 과제 미제출이 되지는 않으니 컨닝이나 인공지능 사용은 최대한 자제해주시고 최선을 다해주세요! 다들 화이팅!!🔥🔥

경사하강법

1. (객관식) 다음 중 SGD(확률적 경사하강법)에 대한 설명으로 올바른 것은?
 - a. N개의 전체 데이터를 모두 계산한 후 파라미터를 한 번에 업데이트한다.
 - b. BGD(Batch Gradient Descent)보다 메모리를 많이 차지한다.
 - c. 한 번에 하나의 데이터를 뽑아 가중치를 업데이트하므로 학습 속도가 빠르지만, 노이즈가 심하고 불안정하다.
2. (주관식) 전체 데이터가 아닌 적당한 크기의 데이터를 활용하여 모델을 업데이트함으로써, SGD의 불안정성과 BGD의 느린 속도를 보완한 경사하강법의 이름은?

Momentum

3. (주관식) Momentum(모멘텀) 기법은 파라미터를 업데이트할 때 기울기를 바로 반영하지 않고, 과거의 이동 방향을 유지하려는 물리적 개념인 []을 도입했다.
4. (객관식) 모멘텀 수식에 등장하는 하이퍼파라미터 α (모멘텀 계수)에 대한 설명으로 가장 적절한 것은?
 - a. 이전 스텝에서 이동하던 방향(속도)을 얼마나 유지할지 결정하며, 보통 0.9 정도의 값을 사용한다.
 - b. 보통 0.001 등 매우 작은 값을 사용하여 이전 관성을 억제한다.
 - c. 값이 클수록 현재 느껴지는 기울기를 많이 반영하여 다음 스텝을 결정한다.